

Client-Centric Consistency in MongoDB

Course project — Consistency Models in Distributed Databases

Group members: Bing Wen Lim · Zhang Xiansheng (Haoran) · Huang Jen-Chien (A0353871X)

Draft compiled from the current repository state. Every result in §5 was produced by running the scripts in this repository against the live 5-node cluster; no result is hand-written. Teammate matriculation numbers to be completed on the cover before submission.

1. Database and architecture

Database. MongoDB 7.0 — a document store whose replica set provides asynchronous primary–secondary replication with tunable read/write semantics. We chose MongoDB because its client-centric guarantees are configured explicitly through `readConcern`, `writeConcern`, `readPreference`, and causally consistent sessions, which maps directly onto the four models under study.

Deployment architecture. A single replica set `rs0` of **five mongod nodes**, one `mongo:7.0` container each, on a shared Docker bridge network `mongo-cluster` (Docker Compose). Member priorities are fixed so the failure experiments are deterministic:

node	host port	priority	role
mongo1	27017	2	always the initial PRIMARY
mongo2	27018	0	secondary (rides the minority side on partition)
mongo3	27019	1	designated failover — wins the majority side
mongo4	27020	0	secondary
mongo5	27021	0	secondary (used as a <i>delayed</i> replica in §4.4)

With these priorities, partitioning `{mongo1, mongo2}` off from `{mongo3, mongo4, mongo5}` deterministically puts the old primary (mongo1) on the losing *minority* side and elects mongo3 on the winning *majority* side.

Software versions. MongoDB 7.0; mongosh (bundled in the image); PyMongo ≥ 4.9 ; Python ≥ 3.11 (run through `uv`); Docker Compose; the partition helper uses the `nicolaka/netshoot` image for in-namespace `iptables`.

Installation / run procedure.

```
./scripts/up.sh          # start 5 containers, initialise rs0, force mongo1 PRIMARY
./scripts/status.sh      # show who is PRIMARY / SECONDARY and replication lag
# ... run experiments (§5) ...
./scripts/down.sh        # stop (add --wipe to also delete data volumes)
```

Client connection detail. The client connects to a *specific* node by its host port with

`directConnection=true`. A `?replicaSet=rs0` URI is **not** usable from the host: the driver rewrites the seed list to the members' container hostnames (`mongo1:27017 ...`), which do not resolve outside the Docker network. Pinning a direct connection is also what lets us route a read to a chosen replica, which the consistency experiments depend on.

2. Consistency configurations explored

Client-centric consistency in MongoDB is governed by three per-operation knobs plus one session-level wrapper.

writeConcern `w` — how many nodes must acknowledge a write before it returns. - `w:1` — acknowledged once the primary has it; **not** necessarily durable. - `w:"majority"` — acknowledged only after a majority (here 3 of 5) has it; durable. - (`j` controls whether the write is flushed to the on-disk journal; `wtimeout` bounds the wait. We fix `j` at its default and set `wtimeout` to keep failed writes from blocking.)

readConcern — what a read is allowed to return. - `local` — the node's most recent data; may be uncommitted and may later roll back. - `majority` — only majority-committed data, which cannot roll back. - (`linearizable` and `snapshot` exist and are stronger; we note them but do not sweep them — see §6.)

readPreference — which node serves a read: `primary`, `secondary`, `nearest`, etc. This is the knob that decides whether a client can be served by a *lagging* replica, and it is central to monotonic reads (§4.4). Where an experiment must read a specific replica we pin it with `directConnection`.

Causally consistent session — `startSession({causalConsistency: true})`. The four client-centric guarantees are defined *over a session*; every experiment runs inside one. This is the wrapper that ties the per-operation knobs together — with it off, even `majority / majority` does not guarantee the four models.

The main experiments sweep the 2×2 grid **readConcern {local, majority} × writeConcern {1, majority}**, with the causal session always on.

3. Predictions (made before running)

From the MongoDB causal-consistency documentation, which tabulates which guarantees hold for each read/write-concern combination. We predicted each cell **before** running anything.

readConcern	writeConcern	RYW	MR	MW	WFR
majority	majority	✓	✓	✓	✓
majority	w:1	✗	✓	✗	✓
local	w:1	✗	✗	✗	✗
local	majority	✗	✗	✓	✗

Reading the columns gives the rule we set out to test:

- **RYW** holds only when both concerns are `majority`.
- **MR** holds iff `readConcern:majority` (read side controls it).
- **MW** holds iff `writeConcern:majority` (write side controls it).
- **WFR** holds iff `readConcern:majority` (read side controls it).

The falsifiable predictions: MW should be broken by `w:1` regardless of `readConcern`; MR and WFR should be broken by `readConcern:local` regardless of `writeConcern`; and on a healthy cluster with no rollback, *nothing* should visibly break — a missing guarantee only costs something when a write or a read is discarded, which requires a real fault.

4. Experiment design and rationale

All violations in MongoDB reduce to one real event: a **rollback**. There is a single primary and one totally ordered oplog, so writes are never reordered; the only way to expose an inconsistency is for an acknowledged/observed write to be **discarded** when a partition heals. Each experiment constructs the specific interleaving that turns a rollback into a violation of one model. No failpoints are used — only real network partitions and real writes.

The partition (`scripts/partition-split.sh`) injects `iptables` DROP rules between the two node groups while leaving every node reachable **from the client**, so we can keep writing to the losing side and can read any replica directly. This is what a plain `docker network disconnect` cannot do (it removes host reachability too), and it is required to manufacture a rollback on demand.

4.1 Read-your-writes (`readConcern` + `writeConcern`)

Write `x=1` in a causal session, read it back. **Rollback path** (`w:1`): the write is acked on the isolated old primary and read back, then discarded on heal → a later read returns the old value. **Divergent-read path** (`local / majority`): the write is durable on the winning side, but a causal `local` read on a minority node whose clock has advanced past the write's timestamp returns stale data without blocking. Only `majority / majority` upholds RYW.

4.2 Monotonic writes (`writeConcern` is the culprit)

One session writes W1 then W2 to two keys. Partition; **W1** (`w:1`) is acked on the doomed old primary; barrier-wait for mongo3's election; **W2** lands on the new primary; heal. W1 rolls back, W2 survives → a later write visible without the earlier one. With `w:majority`, W1 cannot reach a majority on the minority side, is refused, and there is nothing to roll back. **MW holds iff `writeConcern:majority`**.

4.3 Writes-follow-reads (`readConcern` is the culprit)

The session **reads** a doomed `w:1` value with the config's `readConcern`, then issues a **dependent write** recording what it saw. With `local` it reads the doomed value and commits a follow-on write that survives the rollback → the write is ordered after a value that no longer exists. With `majority` the read returns only committed data (never the doomed value), so no dependent write is issued. **WFR holds iff `readConcern:majority`**.

4.4 Monotonic reads — two designs

Basic (network partition). Read a doomed `w:1` value, let it roll back, read again: `local` reads 1 then 0 (goes backwards); `majority` reads 0 both times.

Advanced (normal operation, no partition). The realistic case: with `readPreference` routing a client to a *lagging* replica, reads go backwards with no fault at all. We make `mongo5` a 10-second delayed secondary and read a fresh secondary (sees the latest version) then `mongo5` (sees an old version) → regression. A causal session carrying the first read's cluster time makes the second read **block until mongo5 catches up**, restoring monotonicity at the cost of latency. This design exercises the third knob (`readPreference`) and covers the *normal-operation* scenario, complementing the partition-based experiments.

Scenario coverage (requirement ⑤)

- **Normal operation** — §4.4 advanced (delayed replica, no fault).
- **Node failure** — every partition run forces a primary step-down and a fresh election (`mongo1` → `mongo3`) mid-experiment.
- **Network partition** — §4.1–4.4 basic all run under an active 2-vs-3 split.

5. Results

All observations match the predictions in §3. Verdict legend: **VIOLATED** = the guarantee was broken; **SAFE/HELD** = upheld; **UNAVAILABLE** = the operation was refused (consistent but not available — not a failure).

5.1 Read-your-writes (`./run.sh read-your-writes <config>`)

readConcern	writeConcern	predicted	observed
majority	majority	holds	SAFE
majority	w:1	fails	VIOLATED
local	w:1	fails	VIOLATED
local	majority	fails	VIOLATED

```
=== RYOW / rollback === (majority/w:1)
write acknowledged:  True
read own write:      1
survived heal (X):    0
verdict:              VIOLATED (acknowledged write rolled back)
```

5.2 Monotonic writes (`./run.sh monotonic-writes <config>`)

readConcern	writeConcern	predicted	observed
majority	majority	holds	SAFE
local	majority	holds	SAFE
majority	w:1	fails	VIOLATED
local	w:1	fails	VIOLATED

```
=== Monotonic-writes / rollback === (majority/w:1)
W1 acknowledged: True (k1)
W2 acknowledged: True (k2)
survived heal: k1=0 k2=1
verdict: VIOLATED (W2 visible, W1 rolled back)
```

MW holds iff `writeConcern:majority`; `readConcern` has no effect — confirmed.

5.3 Writes-follow-reads (`./run.sh writes-follow-reads <config>`)

readConcern	writeConcern	predicted	observed
majority	majority	holds	SAFE
majority	w:1	holds	SAFE
local	w:1	fails	VIOLATED
local	majority	fails	VIOLATED

```
=== Writes-follow-reads / doomed read === (local/w:1)
session read k1: 1
dependent W2 issued: True (recorded saw=1)
survived heal: k1=0 k2=1
verdict: VIOLATED (W2 followed a read of k1=1 that rolled back)
```

WFR holds iff `readConcern:majority`; `writeConcern` has no effect — confirmed.

5.4 Monotonic reads (notebook: `experiments/notebooks/monotonic_reads.ipynb`)

Basic (partition + rollback):

readConcern	predicted	observed
local	fails	VIOLATED (read 1, then 0)
majority	holds	SAFE (0 both times)

Advanced (normal operation, delayed replica):

mode	observed	note
naive (no session, <code>local</code>)	VIOLATED	fresh secondary → 6, mongo5 → 0 (backwards)
causal session (<code>majority</code>)	SAFE	mongo5 read <i>blocked</i> 8.1 s until it caught up, returned 6

The causal-session read blocking for 8.1 seconds is the key observation: causal consistency does not make a stale replica fresh — it makes the read wait. MR holds iff `readConcern:majority` (or an equivalent causal gate); confirmed.

6. Discussion

Observations vs. predictions. Every cell of the §3 prediction table was reproduced. MW is broken exactly by `w:1` and fixed by `w:majority`, independent of `readConcern`; MR and WFR are broken exactly by `readConcern:local` and fixed by `majority`, independent of `writeConcern`; RYW needs both. The two underlying mechanisms — a write-concern rollback and a read-concern divergence — account for all failures.

A prediction that first appeared to fail (and why the docs were right). An early version of the checker counted a *failed* write as an ordering violation: during an election a `w:majority` W1 fails while W2 succeeds, leaving W2 "ahead" of a W1 that never happened. This produced violations in the cells the documentation marks safe. Once the checker only counts a gap when the missing write was actually *acknowledged*, the false positives vanished and the real `w:1` violation appears only when a failover lands between the two writes. The first experiment contradicted the documentation; the documentation was right and the measurement was wrong.

Limitations. - *Timing.* The rollback violations need a real failover window (old primary still writable while the new one is elected). We wait for the election as an event rather than sleeping a guess, which is reliable, but a run can still return INCONCLUSIVE and need a re-run. - *Determinism aids.* Fixed member priorities and (for the RYW divergent-read case) an inflated `electionTimeoutMillis` widen a real-but-rare window so it is catchable. MongoDB itself is unmodified — no failpoints. - *Single-key / single-session.* Experiments demonstrate that each violation *exists* under the predicted config; `experiments/trials.sh` repeats a config N times to report a rate rather than a single anecdote. - *Delayed-replica visibility.* A `secondaryDelaySecs` member is hidden from `hello().hosts`, so ordinary `readPreference=secondary` routing never selects it; §4.4 pins it with a direct connection to make the lagging read deterministic. - *Scope.* We swept `readConcern × writeConcern`; `linearizable / snapshot` read concerns and multi-document transactions are stronger settings we did not test.

7. References

1. MongoDB Manual — *Causal Consistency and Read and Write Concerns*.
<https://www.mongodb.com/docs/manual/core/causal-consistency-read-write-concerns/>
2. MongoDB Manual — *Read Concern, Write Concern, Read Preference, Replica Set Elections and Rollbacks*.
3. Terry, D. B., Demers, A. J., Petersen, K., Spreitzer, M. J., Theimer, M. M., & Welch, B. B. (1994). *Session Guarantees for Weakly Consistent Replicated Data*. Proc. PDIS.

AI usage disclosure. AI assistance (Claude Code) was used to help scaffold the Docker/replica-set setup, draft and debug the experiment scripts and the checker, design the failover timing barriers, and

draft this report. All experiments were executed by the team against the live cluster and every reported result was produced and verified by running the code in this repository.